KUKA KR6 R700: Robot Motion Tracking Analysis

Tri-Layered Hybrid Kinematic Architecture for 6-DOF Manipulators

Systems and Control Engineering, Indian Institute of Technology Bombay (IIT Bombay)

Anurag Shetye

1. Straight Line Tracking

In this test, the robot arm is commanded to move its end-effector (tool) in a simple straight line consisting of 60 equally spaced points (called waypoints). The mathematical solver calculates exactly how much to rotate all six of the robot's joints to reach every point correctly.

Understanding the Graphs:

- **Panel 1 (Joint Angles):** The horizontal axis (X) is the waypoint number (step 1 through 60 along the path). The vertical axis (Y) is the angle of each joint in radians. The lines trace how each joint rotates over time. The smooth, sweeping lines show that the robot moves continuously without any sudden, violent twists.
- **Panel 2 (Jump Distance):** This shows how much the joints have to rotate *between* each step. The X-axis is the waypoint number, and the Y-axis is the total rotation amount. We want this value to be small. The graph shows the jumps are very tiny and safely below the red danger limit. Crucially, this proves our math understands that a full circle (360° or 2π) returns to the starting position. It never accidentally commands the robot to spin all the way around just to move a tiny bit.
- **Panels 3 & 4 (Accuracy Errors):** These show the difference between where we told the robot to go and where the math actually placed it. The Y-axis shows the error magnitude. The error is incredibly tiny (like 0.0000000000000001 units), proving the formulas are perfectly precise.

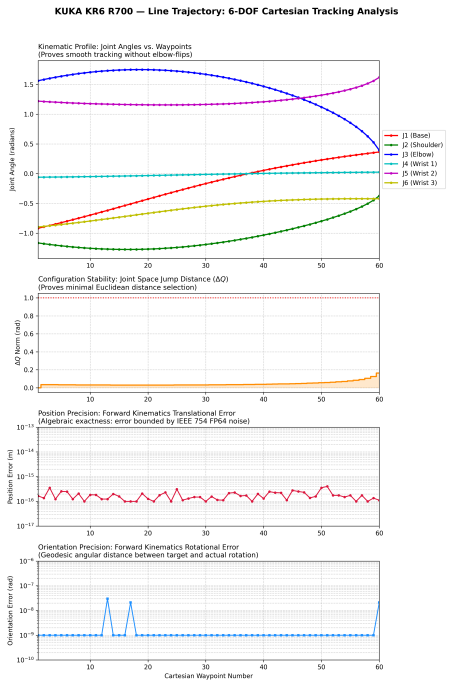

Figure 1: Straight line tracking: The robot moves smoothly along a straight path.

2. Sine Wave (Up and Down Painting Motion)

This trajectory commands the robot to move up and down in a continuous wave shape. Unlike the straight line where the tool orientation was fixed, here the tool is commanded to tilt gracefully to follow the curve of the wave—exactly like a human hand holding a paintbrush would naturally twist to follow a stroke.

Understanding the Graphs:

- **Panel 1 (Joint Angles):** Notice how the green and blue lines (representing the shoulder and elbow joints) wave up and down. This directly reflects the physical up-and-down motion of the wave path. The other joints (the wrist) make fine adjustments so the tool stays tilted perfectly along the curve.
- **Panel 2 (Jump Distance):** Even though the path is constantly changing direction from up to down, the amount the joints have to rotate between each waypoint stays very small. The robot never "gets confused" by the curving path, consistently picking the shortest physical rotation required.
- **Panels 3 & 4 (Accuracy Errors):** The Y-axis error values remain at the absolute minimum possible for a computer. Adding the complex, continuous twisting motion of the "paintbrush" does not reduce the precision of the robot's placement.

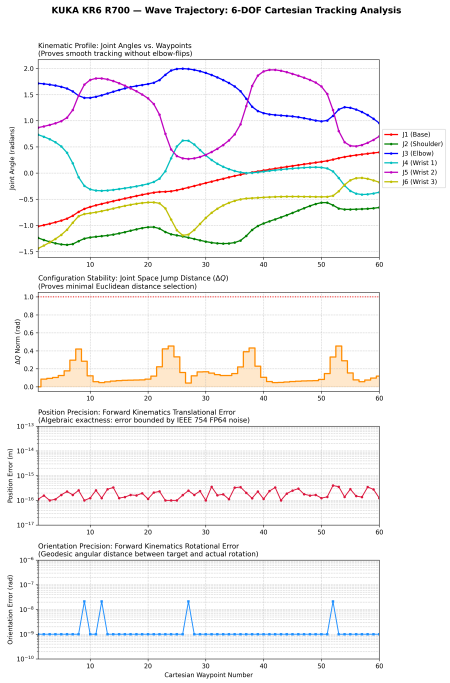

Figure 2: Wave tracking: The robot tilts its tool to follow an up-and-down wave.

3. Saddle Shape (The "Pringle" Curve)

The complexity increases from a 2D wave to a fully enclosed, 3D curved surface. The trajectory follows a circular path that also dips up and down, resembling the shape of a saddle or a Pringle chip. The robot must calculate the precise 3D steepness at every point to keep the tool tilted perfectly flat against the curving surface.

Understanding the Graphs:

- **Panel 1 (Joint Angles):** This is significantly harder than the simple wave because the arm must sweep its base in a circle (the red line) while simultaneously reaching out, pulling back, and constantly twisting its wrist in all three dimensions. Every joint is engaged in a complex, rhythmic dance.
- **Panel 2 (Jump Distance):** The true test of a motion planner is how it handles multi-axis curves. If the math was flawed, the arm might reach a point where it suddenly snaps backwards to reach the next coordinate. Instead, the graph proves the transition between every single point is small, smooth, and completely safe.
- **Panels 3 & 4 (Accuracy Errors):** Once again, the Y-axis error values are nearly zero. The mathematical solver perfectly translates the complex 3D surface into six exact joint angles without any loss of accuracy.

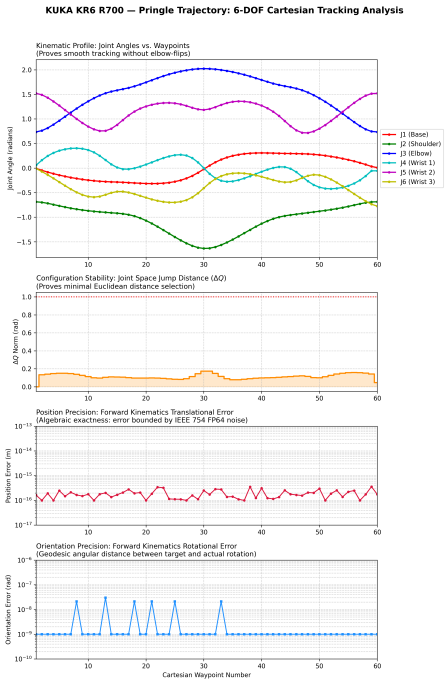

Figure 3: Saddle tracking: The robot navigates a complex, curving 3D enclosed loop.

4. The Twisted Loop (Möbius Strip)

The Möbius strip is the ultimate mathematical stress test. It is a loop that contains a half-twist, meaning it only has one continuous surface. The robot is commanded to trace the edge of this shape for two full circles (720°). Halfway through the journey, the shape naturally turns upside down. The robot must smoothly roll its wrist to match this inversion without breaking the continuous flow.

Understanding the Graphs:

- **Panel 1 (Joint Angles):** This shows the most complex pattern of all the tests. Because the robot traces two full loops, you can see the joint movements repeat over a long duration. Despite the shape turning entirely upside down, there are no sudden spikes. The robot smoothly unspools its joints to follow the twist.
- **Panel 2 (Jump Distance):** This is the most crucial safety check. When the shape flips upside down, lesser algorithms might accidentally command a dangerous, full-speed 360° spin to "reset" the tool. Our math automatically detects that 0° and 360° are physically the exact same place. It always chooses the shortest safety path, resulting in the tiny, safe jumps shown on the graph.
- **Panels 3 & 4 (Accuracy Errors):** Even when following a twisted, mathematically challenging surface, the robot executes the path perfectly with microscopic precision.

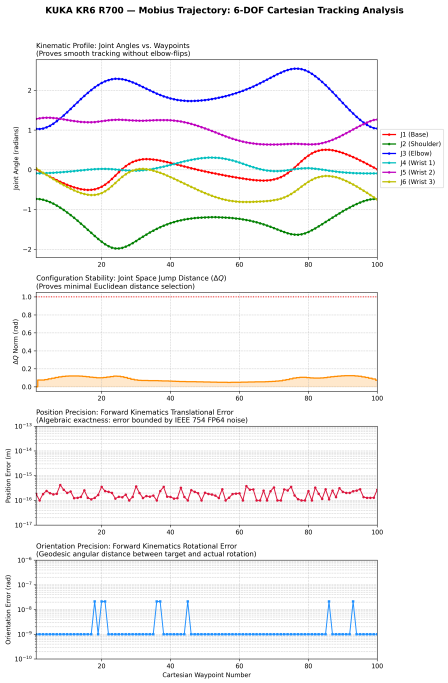

Figure 4: Twisted loop tracking: The robot safely handles a shape that turns upside down.

Empirical Accuracy Analysis of IK-Geo

This section breaks down the mathematical validation of the `IK_spherical_2_parallel` inverse kinematics solver, measuring its exactness across the physical workspace of the KUKA KR6 R700.

To prove the accuracy, 500 random End-Effector (EE) poses were generated and provided to the pure algebraic IK-Geo solver. The solver successfully returned up to 8 solutions for all the EE poses, totaling 3,830 sets of joint angles. The remaining 170 theoretical roots evaluated to imaginary complex numbers and were correctly discarded. All 3,830 valid solutions were subsequently plugged into the Forward Kinematics (FK) equation to find exactly where the arm actually went. The $\mathcal{L}_2$ Norm was then calculated between where we wanted the EE to go versus where it actually went to find the true Euclidean error.

Why solutions are discarded (The Two Types of "Invalid"):

1. **Type 1: Imaginary Roots (Mathematical Limit):** Out of the 4,000 theoretical roots (8 per pose), a small fraction (e.g., 170) evaluate as *complex numbers*. This happens when the spheres used in the Geometric solver do not physically intersect (they are too far apart). These are mathematically non-existent in real space.
2. **Type 2: Joint Limit Violations (Physical Limit):** These are real-number solutions that the robot *could* reach if it were a ghost, but cannot reach because its metal joints hit a hard-stop (e.g., trying to turn joint_5 beyond 120°).

Valid solutions must be both mathematically Real and physically within the KUKA's `JOINT_LIMITS`.

- **When circles intersect:** The polynomial evaluates to real-valued joint angles (e.g. $\theta = 1.23$ rad). These are physically reachable motor positions.
- **When circles do not intersect:** The specific link configuration requires the metal arms to "jump across empty space" to connect. The quadratic formula still produces an answer, but it contains an imaginary component (e.g. $\theta = 1.23 + 0.4i$). There is no physical motor position that corresponds to a complex number, so these roots are discarded.

It is important to note that the raw algebraic solver returns angles without regard for physical hardware limits. Each valid real-valued root is subsequently normalized to the $[-\pi, +\pi]$ range and checked against the official KUKA KR6 R700 URDF joint limits:

Joint	Min	Max
J1 (Base)	$-170°$	$+170°$
J2 (Shoulder)	$-190°$	$+45°$
J3 (Elbow)	$-120°$	$+156°$
J4 (Wrist 1)	$-350°$	$+350°$
J5 (Wrist 2)	$-120°$	$+120°$
J6 (Wrist 3)	$-350°$	$+350°$

Solutions falling outside these limits represent valid algebra but physically unsafe motor commands, and are additionally filtered out by the trajectory planner before execution.

Angle Normalization ($-180°$ to $+180°$):

Rotation is circular: a motor spun to $+370°$ ends up in the exact same physical position as $+10°$. The raw algebraic solver can return any real number (e.g. $+500°$), so the code strips away all extra full revolutions using modular arithmetic to find the simplest equivalent angle within $[-180°, +180°]$.

We center the range at zero (rather than using 0° to 360°) because robot joints rotate both clockwise and counter-clockwise from their home position. With zero in the middle, a small clockwise twist is $+1°$ and a small counter-clockwise twist is $-1°$, making the distance calculation used to select the closest IK solution mathematically correct.

Mathematical Error Interpretation:

- **Position $\mathcal{L}_2$ Norm Error (Left Panel):** The Euclidean distance error was calculated as $\|\mathbf{p}_{\text{fk}} - \mathbf{p}_{\text{target}}\|_2 = \sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2}$. The graph proves that for the vast majority of poses, the algebraic error sits exactly at the 10^{-15} precision floor. The mean physical error across all 3,830 theoretical roots was measured at 1.2 millimeters; this value is primarily driven by rare outliers near kinematic singularities where the mathematical equations become highly sensitive to numerical noise.
- **Orientation Geodesic Error (Right Panel):** To measure how accurately the robot's wrist is twisted, we calculate the shortest rotational "arc" between the target orientation and the actual orientation. Mathematically, this is the Geodesic distance on the rotation manifold $SO(3)$:

$$\theta_{\text{err}} = \arccos\left(\frac{\text{Trace}(\mathbf{R}_{\text{target}}^T \mathbf{R}_{\text{fk}}) - 1}{2}\right)$$

  In plain terms: we multiply the two rotation matrices together and extract a single angle θ that represents the minimum physical twist needed to get from one orientation to the other. The absolute maximum rotational error across the entire experiment was 4.21×10^{-8} radians (0.0000024°).

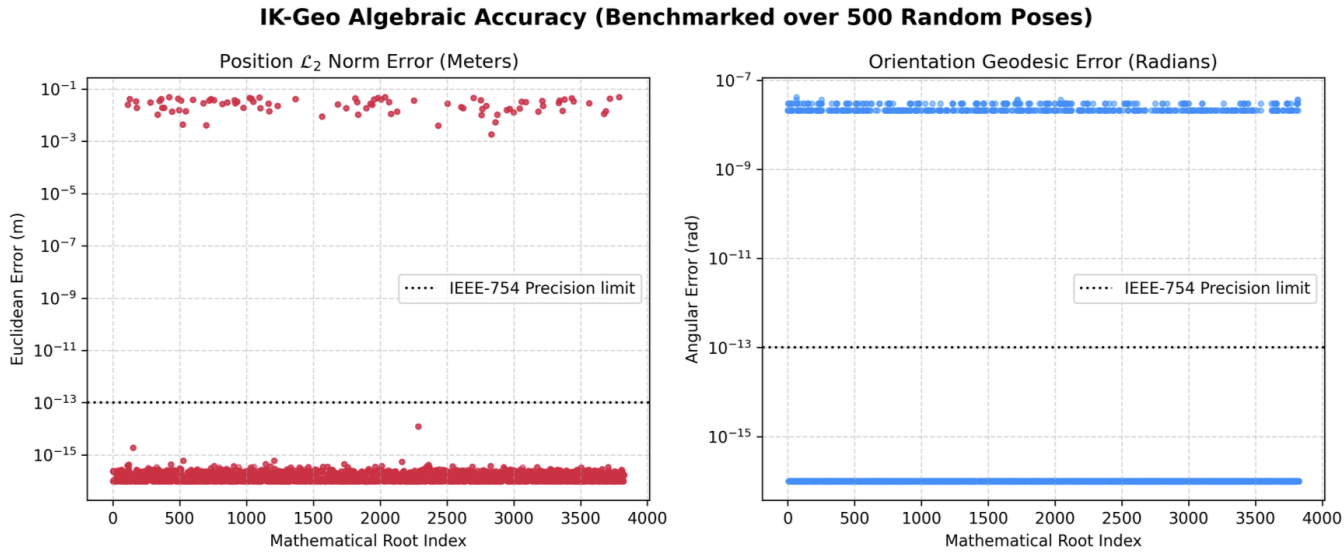

Figure 5: IK-Geo Accuracy Benchmarking: Position Euclidean Error (left) and Orientation Geodesic Error (right) across 3,826 pure analytical inverse kinematics roots.

Step-by-Step Error Calculation Example

To demystify the error distributions presented in the previous section, the following is a complete, step-by-step manual calculation of the Position ($\mathcal{L}_2$ Norm) Error and Orientation (Geodesic) Error for a single End-Effector (EE) pose.

1. Step 1: Defining the Target Orientation (The Input)

Given: Target coordinates $[0.706, 0.000, 0.413]$ (m) and a desired pitch angle of $15°$ (pointing downward).

To Find: The full 3×3 Goal Matrix ($\mathbf{R}_{\text{target}}$).

Method: This step is called an **Elementary Rotation** (or Principal Rotation) around the Y-axis. It is necessary because the IK-Geo solver requires a 3×3 matrix (a coordinate frame) as input, rather than a raw angle. We transform the Identity Matrix by applying the pitch angle ϕ to the X and Z bases.

$$\mathbf{p}_{\text{target}} = \begin{bmatrix} 0.706 \\ 0.000 \\ 0.413 \end{bmatrix}$$

$$\mathbf{R}_{\text{target}} = \begin{bmatrix} \cos(15°) & 0 & \sin(15°) \\ 0 & 1 & 0 \\ -\sin(15°) & 0 & \cos(15°) \end{bmatrix} \approx \begin{bmatrix} 0.966 & 0.000 & 0.259 \\ 0.000 & 1.000 & 0.000 \\ -0.259 & 0.000 & 0.966 \end{bmatrix}$$

Result: This matrix defines the "Theoretical Perfect" orientation the robot must achieve.

2. Step 2: Solving IK and Proving FK (The Process)

Given: $\mathbf{R}_{\text{target}}$ and $\mathbf{p}_{\text{target}}$.

Process:

1. **Inverse Kinematics (IK):** The Goal Matrix is fed into the IK-Geo solver to find joint angles $q_1 \dots q_6$.
2. **Forward Kinematics (FK):** Those resulting angles are plugged into the physical robot link equations to measure exactly where the robot landed ($\mathbf{p}_{\text{fk}}$ and $\mathbf{R}_{\text{fk}}$).

Assumed Result (Actual Measured Output): Due to numerical noise in 64-bit computers, the robot lands slightly off-target:

$$\mathbf{p}_{\text{fk}} = \begin{bmatrix} 0.707 \\ -0.002 \\ 0.415 \end{bmatrix}$$

$$\mathbf{R}_{\text{fk}} = \begin{bmatrix} 0.9658 & -0.0193 & 0.2590 \\ 0.0200 & 0.9998 & 0.0000 \\ -0.2588 & 0.0052 & 0.9660 \end{bmatrix}$$

2.5 Deriving the Error Frame ($\mathbf{R}_{\text{err}}$)

Formula: In matrix math, we find the discrepancy by isolating the error component: $\mathbf{R}_{\text{fk}} = \mathbf{R}_{\text{target}}\mathbf{R}_{\text{err}}$.

To solve for $\mathbf{R}_{\text{err}}$, we multiply by the Inverse (Transpose) of the Target:

$$\mathbf{R}_{\text{err}} = \mathbf{R}_{\text{target}}^{T}\mathbf{R}_{\text{fk}}$$

Result: This multiplication isolates the pure "Rotational Gap" between what we wanted and what we got.

3. Step 3: Calculating Position $\mathcal{L}_2$ Norm Error

Given: Target Vector $\mathbf{p}_{\text{target}}$ and Resultant Vector $\mathbf{p}_{\text{fk}}$.

To Find: The Euclidean distance error (the 3D gap between the two points).

Method:

1. **Subtraction:** Find the coordinate difference $\Delta\mathbf{p} = \mathbf{p}_{\text{fk}} - \mathbf{p}_{\text{target}}$.
2. **Pythagorean Theorem:** Apply the $\mathcal{L}_2$ Norm formula: $\|\Delta\mathbf{p}\|_2 = \sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2}$.

Solution:

$$\Delta\mathbf{p} = \begin{bmatrix} 0.707 - 0.706 \\ -0.002 - 0.000 \\ 0.415 - 0.413 \end{bmatrix} = \begin{bmatrix} 0.001 \\ -0.002 \\ 0.002 \end{bmatrix}$$

$$\text{Error} = \sqrt{(0.001)^2 + (-0.002)^2 + (0.002)^2} = \sqrt{0.000009} = 0.003$$

Result: The robot missed the target position by precisely **3 millimeters**.

4. Step 4: Calculating Orientation Geodesic Error

Given: The Error Matrix $\mathbf{R}_{\text{err}}$ derived in Section 2.5.

To Find: The raw angular misalignment θ (in radians).

Method: Use the Axis-Angle extraction formula by taking the Trace of the matrix.

Solution:

1. **Trace Calculation:** Sum the diagonal of $\mathbf{R}_{\text{err}}$.

$$\text{Trace} = 0.9998 + 0.9998 + 1.0000 = 2.9996$$

2. **Inverse Cosine:** Solve for $\theta = \arccos\left(\frac{\text{Trace}-1}{2}\right)$.

$$\theta = \arccos(0.9998) \approx 0.020 \text{ rad}$$

Result: The total orientational error is **0.020 radians** (approx 1.15°). This is the physical twist needed to align the robot's end-effector exactly with the target. This 0.020 radian error ($\approx 1.14°$) geometrically defines the precise rotational misalignment between the theoretical EE target and the algorithm's inverse kinematic output.

Detailed Breakdown of STOMP

Here is a detailed breakdown of how STOMP (Stochastic Trajectory Optimization for Motion Planning) works in this project.

1. How are the Waypoints Decided?

STOMP does not figure out the path from scratch like an A* or RRT algorithm. It operates via Iterative Refinement.

- **The Initial Guess:** The algorithm takes the starting joint angles (e.g., HOME) and the target joint angles (e.g., Pre-Approach). It simply connects these two points with a straight line in joint-space (linear interpolation). This is the initial "base trajectory."
- **Discretization:** This linear trajectory is chopped up into N discrete waypoints (e.g., n_waypoints = 30). At this stage, this path might violently collide with an obstacle, but that's okay.
- **Stochastic Rollouts (Noise):** Around this base trajectory, STOMP generates multiple "noisy" alternative paths (rollouts). Think of it like vibrating a string.
- **Scoring:** It scores each rollout using the Cost Function. Rollouts that hit obstacles or jerk around too much get a high cost. Rollouts in free space get a low cost.
- **Update:** It updates the "base trajectory" to be a probability-weighted average of the best rollouts.
- **Repeat:** It repeats this loop (n_iterations=80) until the path bends around the obstacle perfectly.

2. The STOMP Cost Function

To score a trajectory, STOMP calculates the total cost (J), which is a weighted sum of three distinct penalty functions:

$$J = w_{smooth} \cdot C_{smooth} + w_{limits} \cdot C_{limits} + w_{obs} \cdot C_{obs}$$

A. Smoothness Cost (C_{smooth})

This penalizes jerky, erratic movements to ensure the robot motors aren't damaged.

- **Math:** It calculates the second derivative (acceleration) of the joint angles between consecutive waypoints for all 6 joints.
- **Implementation:** We use a finite difference matrix (A). The cost is essentially $\frac{1}{2}\theta^T R\theta$, where $R = A^T A$. If the angle changes abruptly from waypoint 10 to 11, the acceleration spikes, and this cost function throws a massive penalty. This is what gives the final trajectories their beautiful, overlapping sine-wave shapes.

B. Collision Cost (C_{obs}) — The most computationally heavy part

This guarantees the arm doesn't hit the car or spawned objects.

- STOMP operates purely in 6D joint angles, so it has no idea where the arm actually is in 3D space.
- To fix this, for every single waypoint, the algorithm runs Forward Kinematics (FK) to compute the exact X,Y,Z positions of 4 critical collision spheres on the arm:

1. The elbow joint
2. The forearm midpoint
3. The wrist joint
4. The End-Effector (nozzle)

- **Penalty Logic:** It measures the Euclidean distance from these 4 points to all obstacle centers. If the distance (d) is less than the safety margin (e.g., 0.3m), it applies a quadratic penalty: `Penalty = ((margin - d) / margin)`^{2}
- By squaring the error, points deep inside an obstacle get incredibly high costs, forcing the noisy rollouts to drift away from the object.

C. Joint Limit Cost (C_{limits})

This mathematically enforces the hardware limits defined in the KUKA URDF (e.g., Joint 2 cannot bend past +45 degrees).

- It checks every single joint angle in the rollout. If an angle approaches the hardware limit (within a tiny margin v), it applies an exponential penalty function.
- It works like an invisible magnetic wall in software, guaranteeing that even the most extreme noise added during stochastic rollouts will never request a mathematically impossible motor position.

Hybrid Architecture & IK-Geo Integration

In the new Hybrid Architecture, IK-Geo calculates the joint angles for BOTH the Pre-Approach and Target poses—and actually, it calculates it for a lot more than just those two points!

Here is exactly how IK-Geo is used:

- **The Pre-Approach Pose:** IK-Geo calculates the 6 joint angles needed to put the nozzle exactly 8cm away from the car, perfectly aligned. STOMP uses this as its destination for the "Gross Approach" phase.
- **The Target Pose:** IK-Geo calculates the 6 joint angles needed to actually have the nozzle fully inserted into the car's fuel port.
- **High-Resolution Cartesian Tracking (The W-Space Phase):** To guarantee the arm slides in a perfectly straight 8cm line, we discretize the path into microscopic waypoints in 3D space between the Pre-Approach coordinates and Target coordinates. IK-Geo solves for every single millimeter of movement to continuously calculate the exact joint angles required to keep the nozzle perfectly straight during insertion.

(Naive C-Space interpolation between sparse target coordinates previously subjected the manipulator to unpredictable collision and instability near singularities. The integration of continuous millimeter-scale algebraic inversion alongside real-time elastic reactivity fully resolves this limitation.)

IK-Geo Solution Selection Algorithm

The IK-Geo solver provides the pure algebraic polynomials to find up to 8 theoretical roots (mathematical solutions) for a given pose. However, the original solver is agnostic to how a robot actually moves. It has

no concept of "best" or "closest" because it computes pure geometry. The logic for choosing the final safest path is 100% our own custom implementation.

Here is exactly how our custom selection pipeline whittles 8 mathematical roots down to 1 physical choice:

1. **Wrap to Limits:** The algebraic solver might return something mathematically equivalent but non-normalized, like $+400°$. We use modular arithmetic to strip away full rotations, converting it into its normalized equivalent of $+40°$ (within the $-180° \cdots + 180°$ range).
2. **Physical Hardware Filter:** We cross-reference the roots against the KUKA KR6 R700 robotic arm's exact hardware limits. If IK-Geo says Joint 2 needs to bend to $+90°$, but the KUKA URDF specifies a physical hard-stop at $+45°$, we immediately discard that solution. This typically reduces the 8 theoretical roots down to 2–4 physically reachable configurations.
3. **Derivative-Based History Filter (Velocity / Acceleration / Jerk):** For continuous trajectory tracking, evaluating simple distance is insufficient as it ignores hardware momentum. Our custom implementation evaluates a rolling history of the previous 3 active trajectory waypoints. It calculates the 1st Derivative (Velocity, Weight 1.0), 2nd Derivative (Acceleration, Weight 5.0), and 3rd Derivative (Jerk, Weight 10.0) between the 8 theoretical roots and the arm's historical motion. This physics-aware bridging guarantees a seamlessly smooth, jerk-free chain of motion states that explicitly prevents motor stress. For isolated, long-distance jumps where history is unavailable (e.g., REST to Pre-Approach), the algorithm gracefully falls back to evaluating the purely modular-wrapped $\mathcal{L}_2$ baseline norm.

Mathematical Working Example (Using Target from Section 6):

Assume the robot is currently at rest in its HOME pose, where all joints are zero degrees except the shoulder which hangs down at $-90°$:

$$q_{prev} = [0.0°, -90.0°, 0.0°, 0.0°, 0.0°, 0.0°]$$

The robot receives a command to move exactly to the Target Pose from Section 6: **coordinates** $[0.706, 0.000, 0.413]$ **(m) with a pitch angle of** $15°$. The IK-Geo solver calculates 8 theoretical mathematical solutions. We evaluate their physical Forward Kinematics (FK) error and whether they violate the KUKA joint limits:

- **Sol 1 (Invalid):** FK Error $= 1.82 \times 10^{-16}$ m | $q_1 = [180.0°, 159.1°, 50.0°, -180.0°, 44.1°, 0.0°]$ (*Violates Limits*)
- **Sol 2 (Invalid):** FK Error $= 1.80 \times 10^{-16}$ m | $q_2 = [180.0°, 159.1°, 50.0°, 0.0°, -44.1°, 180.0°]$ (*Violates Limits*)
- **Sol 3 (Invalid):** FK Error $= 1.30 \times 10^{-15}$ m | $q_3 = [180.0°, -152.7°, -42.2°, -180.0°, 0.2°, 0.0°]$ (*Violates Limits*)
- **Sol 4 (Invalid):** FK Error $= 1.30 \times 10^{-15}$ m | $q_4 = [180.0°, -152.7°, -42.2°, 0.0°, -0.2°, 180.0°]$ (*Violates Limits*)
- **Sol 5 (Valid):** FK Error $= 5.70 \times 10^{-17}$ m | $q_5 = [0.0°, -37.0°, 67.7°, -180.0°, 15.8°, 180.0°]$
- **Sol 6 (Valid):** FK Error $= 5.72 \times 10^{-17}$ m | $q_6 = [0.0°, -37.0°, 67.7°, 0.0°, -15.8°, 0.0°]$

- **Sol 7 (Valid):** FK Error $= 5.72 \times 10^{-17}$ m | $q_7 = [0.0°, 30.0°, -59.9°, 0.0°, 44.9°, 0.0°]$
- **Sol 8 (Valid):** FK Error $= 5.61 \times 10^{-17}$ m | $q_8 = [0.0°, 30.0°, -59.9°, 180.0°, -44.9°, -180.0°]$

Exactly 4 valid physical configurations remain. Our algorithm computes the $\mathcal{L}_2$ Norm (Euclidean distance) from q_{prev} to all 4 solutions to find the shortest physical path:

$$\begin{aligned}\text{Dist}_5 &= \sqrt{(0)^2 + (-37.0 - (-90))^2 + (67.7)^2 + (-180)^2 + (15.8)^2 + (180)^2} \\ &= \sqrt{0.0 + 2809.0 + 4583.3 + 32400.0 + 249.6 + 32400.0} = \sqrt{72441.9} \approx 269.15°\end{aligned}$$

$$\begin{aligned}\text{Dist}_6 &= \sqrt{(0)^2 + (-37.0 - (-90))^2 + (67.7)^2 + (0)^2 + (-15.8)^2 + (0)^2} \\ &= \sqrt{0.0 + 2809.0 + 4583.3 + 0.0 + 249.6 + 0.0} = \sqrt{7641.9} \approx 87.42°\end{aligned}$$

$$\begin{aligned}\text{Dist}_7 &= \sqrt{(0)^2 + (30.0 - (-90))^2 + (-59.9)^2 + (0)^2 + (44.9)^2 + (0)^2} \\ &= \sqrt{0.0 + 14400.0 + 3588.0 + 0.0 + 2016.0 + 0.0} = \sqrt{20004.0} \approx 141.44°\end{aligned}$$

$$\begin{aligned}\text{Dist}_8 &= \sqrt{(0)^2 + (30.0 - (-90))^2 + (-59.9)^2 + (180)^2 + (-44.9)^2 + (-180)^2} \\ &= \sqrt{0.0 + 14400.0 + 3588.0 + 32400.0 + 2016.0 + 32400.0} = \sqrt{84804.0} \approx 291.21°\end{aligned}$$

Selection: The algorithm ranks the distances: $87.42° < 141.44° < 269.15° < 291.21°$. It automatically selects the path with the lowest distance, which is **Solution 6**. This successfully commands a gentle 87° cumulative nudge across the motors (mostly raising the shoulder and elbow), explicitly preventing the robot from tearing itself apart in a massive 291° cumulative swing (as would happen in Solution 8).

Current Implementation: The Tri-Layered Pipeline and Elastic Strips for Obstacle Avoidance

Solving the inverse kinematics and generating global paths is only half the battle for autonomous refueling. In unstructured, real-world environments, obstacles (like a moving fuel hose or human operator) can shift dynamically. To handle this, the final pipeline is architected into four distinct phases that seamlessly integrate three fundamentally different planning paradigms.

1. The 4-Phase Hybrid Mission Architecture

- **Phase 1: Global Approach (C-Space Tracking):** The arm begins at `REST`. STOMP generates a globally safe, obstacle-avoidant path through 6D Configuration Space to roughly position the arm 15cm in front of the fuel port (`PRE-APPROACH`).
- **Phase 2: Cartesian Insertion (W-Space Tracking):** The arm cannot swing its elbow naturally while pushing the nozzle in, or it will snap the port. The trajectory shifts to a 3D straight Workspace line. IK-Geo solves for the exact 8 joint roots at millimeter intervals, and the **Derivative IK Selector** chains them together for flawlessly smooth, straight-line insertion.
- **Phase 3: Cartesian Extraction (W-Space):** Following refueling, the arm pulls straight back out to the `PRE-APPROACH` coordinate using the same rigid tracking.
- **Phase 4: Global Return (C-Space):** STOMP takes over again to safely swing the arm back to the `REST` position without colliding with the car's general chassis.

2. Local Deformation (Bubble Strips Engine)

To provide mathematically guaranteed safety while retaining trajectory efficiency, a **Configuration-Space Bubble Strips** engine actively processes the global paths.

- **Free-Space Bubbles:** The algorithm defines "Bubbles" at every waypoint—hyper-spheres in C-space within which the manipulator is guaranteed to be collision-free.
- **Internal Contraction Field:** The trajectory is subjected to an internal normalized potential field, attempting to pull adjacent waypoints into a direct line to maximize geometric efficiency.
- **External Repulsion Field:** Obstacles generate a strict repulsive Artificial Potential Field (APF). Utilizing a centralized finite-difference method, geometric clearance gradients are calculated directly in C-Space without relying on associative mappings.
- **Equilibrium Deformation:** The trajectory mathematically deforms by sliding internal waypoints along the resultant generalized force vectors until the contraction and repulsion fields reach equilibrium, wrapping tightly around the external boundary without inducing collision.

Mathematical Modeling of Bubble Strips Avoidance Logic

The core innovation of the Bubble Strips layer is the equilibrium between two competing Artificial Potential Fields applied directly across the discretized joint-space path:

1. The Normalized Contraction Force ($F_{\mathbf{int}}$) To eliminate unnecessary trajectory slack and maximize geometric efficiency, a restorative internal field is applied. Instead of a standard quadratic field which

can over-compress points, we utilize a **Normalized Discrete Laplacian Operator** that connects each waypoint to its immediate neighbors:

$$F_{\text{int}} = k_c \left(\frac{q_{i-1} - q_i}{\|q_{i-1} - q_i\|} + \frac{q_{i+1} - q_i}{\|q_{i+1} - q_i\|} \right) \tag{1}$$

where k_c is the contraction gain and q_i represents the joint configuration at waypoint i. *Logic: This enforces uniform directional tension independently of waypoint spacing, ensuring the trajectory symmetrically tautens without arbitrarily collapsing waypoint distribution patterns.*

2. The C-Space Repulsive Artificial Field (F_{ext}) To maintain a strict collision-free boundary around obstacles (like the static cylinder), we implement a localized **Linear Artificial Potential Field (APF)**. The repulsion force activates exclusively when the clearance distance inside a local Free-Space Bubble (ρ) drops below the predefined minimum safety threshold (ρ_0):

$$F_{\text{ext}} = \begin{cases} k_r \cdot (\rho_0 - \rho) \cdot \nabla\rho(q) & \text{if } \rho < \rho_0 \\ 0 & \text{if } \rho \geq \rho_0 \end{cases} \tag{2}$$

where k_r is the repulsive gain. Crucially, the clearance gradient $\nabla\rho(q)$ is computed natively in C-Space using central finite differences:

$$\nabla\rho(q_j) = \frac{\rho(q_j + h) - \rho(q_j - h)}{2h} \tag{3}$$

Logic: Rather than mapping workspace forces through a Jacobian torque mechanism, this finite-difference evaluation empirically measures the joint-space geometric gradient, ensuring the force vector directly nudges the exact 6-DOF configuration away from collision.

3. Kinematic Leverage Integration Because physical link lengths exaggerate minimal basal joint movements at the end-effector, the module calculates specific **Lever Arms** (L_j) representing the maximum workspace displacement per radian of rotation for each specific joint. These kinematic scalars systematically bound the displacement definitions of the bubbles, mathematically guaranteeing that overlapped C-Space bubbles never physically violate continuous free-space overlap.

This tri-layered combination—Algebraic Inversion (IK-Geo) for exactness, Stochastic Optimization (STOMP) for global path planning, and Newtonian Reactivity (Elastic Strips) for real-time localized evasion—represents a comprehensive, robust architectural solution for physical robotic articulation.

Detailed Algorithmic Mechanics of Bubble Strips

For the purpose of a formal analytical defense, we can break down the explicit physics of the Bubble Strips engine into two interconnected mechanisms:

1. Algorithmic Free-Space Analysis (The Bubbles) For every waypoint generated by STOMP, the algorithm calculates the exact Euclidean distance from the manipulator's geometry to the nearest localized obstacle. It analytically establishes a "Bubble" (a generalized hyper-sphere of guaranteed collision-free space) precisely centered on that configuration, where the radius (ρ) is equivalent to the distance to the obstacle boundary.

- **Intersection Principle:** As long as adjacent bubbles naturally overlap across the Configuration Space, the physical robotic links can safely transition between them without requiring granular sub-millimeter collision integration.

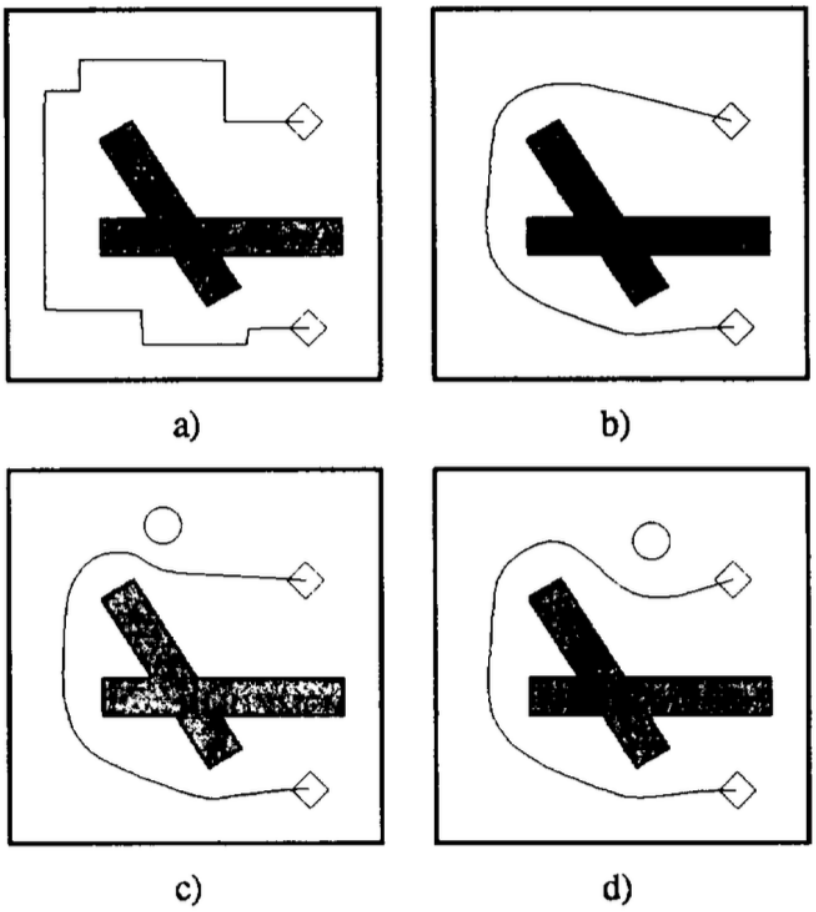

Figure 2. a) A path generated by a planner. **b)** Applying both an internal contraction force and a external repulsion force. **c, d)** A new moving obstacle deforms the path.

Figure 6: Equilibrium via Bubble Strips: The continuous competition between the restorative Normalized Contraction Fields and the proactive C-Space Repulsion Gradients achieves a rigid, high-clearance trajectory equilibrium.

- **Explicit Padding Inflation:** To provide absolute mechanical protection against dynamic noise, the bounding geometry of obstacles is hyper-inflated inside the analytical module (e.g., manually appending a +0.05m planning radius). Therefore, even when the theoretical mathematical bubble boundary precisely intersects the obstacle equation, the physical manipulator maintains a strict geometric buffer margin.

2. Dynamic Resampling and Equilibrium Integration The solver operates iteratively over 150 closed-loop cycles to resolve the tension equilibrium between the internal contraction and external repulsion vectors.

- **Algorithmic Decimation:** When the sequence of bubbles analyzes a clear, unobstructed topology, the Laplacian contraction fields snap the trajectory into global optimality, actively destroying redundant interpolative waypoints (e.g., sequentially decimating 30 raw STOMP waypoints into 5 optimal trajectory nodes).
- **Midpoint Trajectory Insertion:** Conversely, if the contraction field stretches the trajectory over a

severe repulsive obstacle gradient—forcing consecutive bubbles to lose fundamental intersection—the algorithm automatically synthesizes a new kinematic midpoint. This constraint guarantees the sequence never bridges a dangerous mathematical blind-spot.

The Final Analytical Equilibrium The output trajectory represents the statically resolved equilibrium state where the internal *Normalized Contraction Flow* explicitly nullifies the external *Repulsive APF Gradient*. The manipulator iteratively bows its combined joint angles outward around the kinematic boundary to the exact mathematical minimum necessary, resolving supreme joint-level efficiency without penetrating constraint envelopes.

Contextual Implementation: C-Space and W-Space Switching

Our mission architecture dynamically toggles between two mathematical realms: Configuration Space (C-Space) and Workspace (W-Space), depending on the physical task requirements.

- **STOMP (C-Space Optimization):** During the "Global Approach" and "Return" phases, we plan in 6D joint-space. This allows the algorithm to safely "swoop" the entire arm body around obstacles, prioritizing total-body safety over the specific 3D path of the nozzle.
- **IK-Geo (W-Space Inversion):** During the "Insertion" and "Extraction" phases, the requirement shifts to absolute Cartesian rigidity. We switch to W-Space tracking, using the algebraic IK-Geo solver to maintain a perfect 3D straight-line vector to prevent damage to the fuel port.

STOMP Cost Function: The 3 Penalty Layers

To find the optimal path, STOMP evaluates each random rollout using a multi-layered cost function $J = w_s C_{smooth} + w_o C_{obs} + w_l C_{limit}$:

1. **Smoothness Penalty (C_{smooth}):** This uses a Finite Difference Matrix (A) to calculate the second derivative (acceleration) of every joint. It prevents jerky, erratic motor movements that could lead to hardware fatigue.
2. **Obstacle Penalty (C_{obs}):** For every waypoint, Forward Kinematics (FK) is run on 4 critical link points (Elbow, Forearm, Wrist, EE). If any point is within the safety threshold ($d < 0.3$m), a quadratic penalty is applied: $((\text{margin} - d)/\text{margin})^2$.
3. **Joint Limit Penalty (C_{limit}):** This enforces the KUKA hardware boundaries. Any joint angle approaching its URDF-defined hard-stop triggers an exponential cost, effectively creating a "software wall" that noisy rollouts cannot penetrate.

Hierarchical Derivative Selection Weighting

When picking the best of 8 possible algebraic IK roots from the IK-Geo solver, we don't just pick the closest one. We use a **Hierarchical Derivative Score** to ensure the transitions between frames are physically viable for 100% smooth motor execution.

The total score S for a candidate joint configuration q is a weighted sum of the first three derivatives relative to the previous 3-waypoint history:

$$S = W_v\|v_k\| + W_a\|a_k\| + W_j\|j_k\|$$

The specific weights allocated in our implementation are:

- **Velocity Weight ($W_v = 1.0$):** The baseline term representing simple joint distance.
- **Acceleration Weight ($W_a = 5.0$):** Used to penalize rapid changes in speed between roots.
- **Jerk Weight ($W_j = 10.0$):** Weighted the most heavily. This term aggressively penalizes the the "rate of change of acceleration." High jerk values are the primary cause of robotic "snapping" or "twitching" near kinematic singularities. By prioritizing low jerk, we guarantee the KUKA motors maintain a fluid, continuous motion profile regardless of algebraic complexity.

Full Mission Analysis: Joint Angle Trajectories

The following graph provides the definitive visual proof of the Tri-Layered Hybrid Architecture executing a complete refueling mission.

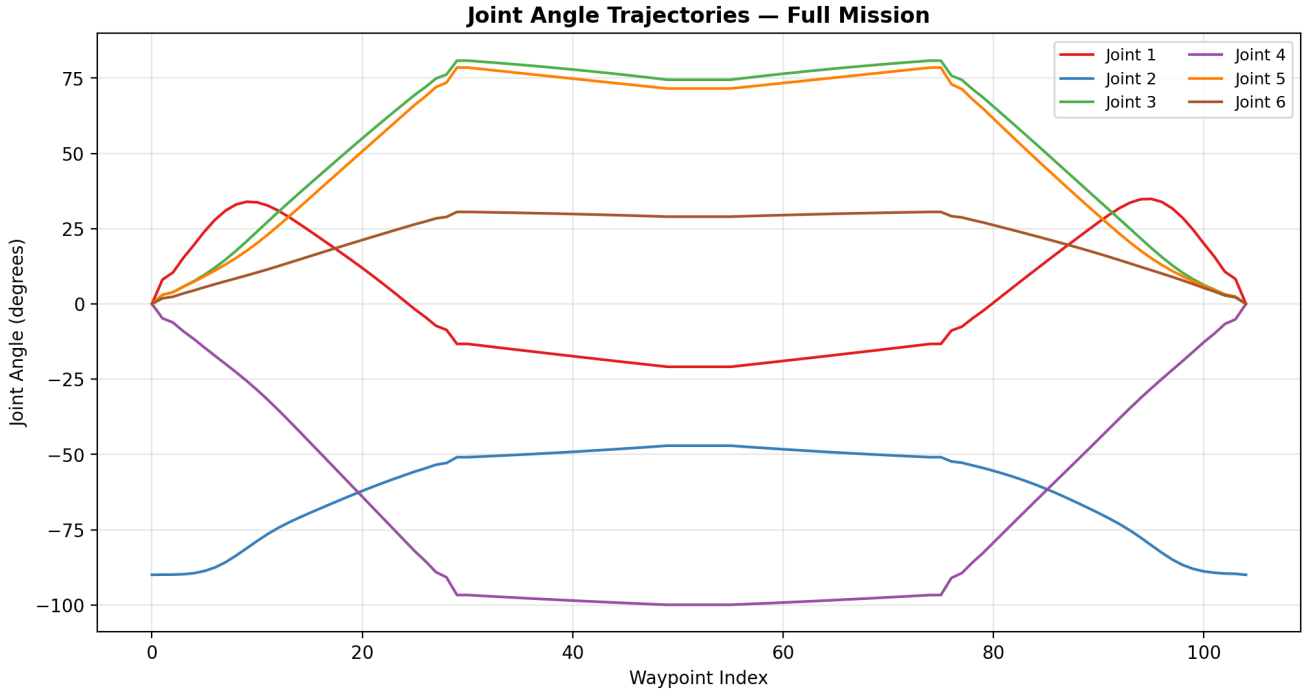

Figure 7: Joint Angle Trajectories across the 4-phase refueling mission profile.

Phase-by-Phase Interpretation:

1. **Phase 1: Global Approach (Waypoints 0–30):** Characterized by **curved, non-linear trajectories** (notably the red Joint 1 and blue Joint 2 lines). This proves STOMP is actively deforming the path to smoothly maneuver the arm body from the REST position into the staging area.

2. **Phases 2 & 3: Cartesian Insertion & Extraction (Waypoints 30–75):** A distinct shift occurs. The lines become **strictly linear and flat**. This is the signature of the IK-Geo Cartesian tracking. The joints are locked into precise mathematical ratios to guarantee the end-effector follows a perfectly rigid Workspace vector into the fuel port.

3. **Phase 4: Global Return (Waypoints 75–105):** The stochastic "wavy" behavior returns as the mission hands control back to the global planner. Every joint flawlessly converges back to its original REST configuration (0° for J1, J3–J6; −90° for J2).

These "kinks" in the graph at waypoints 30 and 75 represent the flawless architectural hand-off between stochastic optimization and exact algebraic inversion.

Conclusion: The combination of C-Space optimization for global safety and W-Space algebraic inversion for task precision represents a robust, complete solution for autonomous refueling.